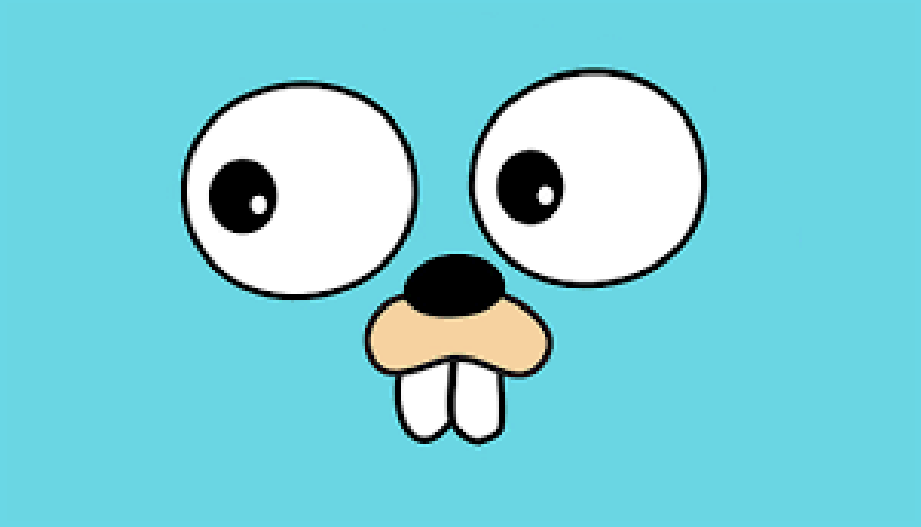

Testing in Go

Discussion Points

- Writing unit tests
- Table-driven tests
- Benchmarking
- Mocking dependencies

Testing in Go

Types of Tests in Go?

Unit Tests

Purpose: Test individual functions or methods in isolation.

Goal: Ensure each unit of code (e.g., function) performs as expected under various scenarios.

Best Use Case: When testing simple functions, logic, or methods that don't rely on external systems.

Example: Testing a single function that calculates an average or performs a basic arithmetic operation.

Integration Tests

Purpose: Verify that multiple components or modules work together correctly.

Goal: Catch issues with interactions between components, such as database queries, API requests, and file I/O.

Best Use Case: When testing functions that rely on external services, databases, or system calls.

Example: Testing a function that retrieves and processes data from a database.

Table-Driven Tests

Purpose: Organize and run multiple test cases with different inputs and expected outputs in a single test.

Goal: Simplify the testing of a function with various scenarios, making the test more readable and maintainable.

Best Use Case: Functions that need to handle different input values or edge cases.

Example: Testing a string manipulation function with various input strings (e.g., empty strings, symbols, long text).

Benchmark Tests

Purpose: Measure the performance of code to identify any potential bottlenecks.

Goal: Optimize function speed and efficiency by analyzing performance metrics.

Best Use Case: Performance-critical code, such as sorting algorithms or data processing functions.

Example: Benchmarking a search algorithm to ensure it meets performance requirements.

Writing unit tests

What is Unit Test?

A unit test is a function that tests a specific piece of code from a program or package. The job of unit tests is to check the correctness of an application, and they are a crucial part of Go

- Early Bug Detection
- Refactoring Confidence
- Quality Assurance
- Documentation

Writing unit tests

Overview of Testing in Go?

Go provides a built-in **testing** package for writing test cases, making it easy to test code without external dependencies. Includes tools for writing unit tests, benchmarks, and example-based tests, all within the Go ecosystem.

Test files in Go should follow the naming convention: ***_test.go**.

Example: If you're testing functions in math.go, the test file should be named math_test.go.

The *_test.go files are ignored during normal builds but are included when running tests with go test.

Writing unit tests

Overview of Testing in Go?

Run all test files in a package using the command

go test ./…

This command automatically detects and executes any function starting with Test in files named *_test.go

Basic Structure of a Test Function?

Each test function typically follows this structure:

Arrange: Set up the data and conditions for the test.
Act: Call the function under test with the prepared data.
Assert: Check the result, using *t.Error()* or *t.Fail()* if the function doesn't behave as expected.

Writing unit tests

Testing Framework Testify?

In Go, the standard library provides the testing package, which is widely used for writing unit tests.

However, to enhance testing capabilities, several third-party packages are commonly utilized

Testify - This is one of the most popular testing frameworks in the Go ecosystem. Testify offers a comprehensive set of tools, including:

Assertions: Simplifies the process of writing test assertions.
Mocking: Provides support for creating mock objects to simulate dependencies.
Suite: Allows grouping of tests and setup/teardown methods.

go get github.com/stretchr/testify

What Are Table-Driven Tests?

A method to structure tests that allows multiple input scenarios to be tested within a single test function. Uses a "table" of test cases, making it easy to add, modify, or expand test scenarios. Preferred style in Go for its readability and maintainability, especially when testing functions with multiple inputs and expected outcomes.

- Reduces Code Duplication
- Improves Readability
- Simplifies Maintenance

Table-Driven Test Structure

Define the test case struct – includes input, expected output, and a description for each case.
Loop through test cases – run the test function on each case within a loop.
Validate results – compare output to expected value, logging errors as needed.

Mocking dependencies

What is Mock?

A mock is an object that simulates the behavior of a real object, allowing you to set expected responses and verify interactions. Mocking a database connection, an HTTP client, or any service that returns data.

Features of Testify Mocks

- Define expectations (e.g., expected method calls and return values).
- Verify that methods are called as expected.

Isolate Dependencies

Focus on Logic

Speed Up Tests

Benchmarking

Writing Benchmarks?

Benchmark tests measure the performance of code, helping identify potential bottlenecks. Useful in optimizing critical parts of the application, especially with heavy computation or frequently called functions.

Benchmark Test Structure

- Benchmark functions in Go start with Benchmark followed by the function name (e.g., BenchmarkFunctionName).
- Each benchmark test accepts a **b *testing.B** parameter.
- The loop for i := 0; i < b.N; i++ runs the code repeatedly to measure time accurately.

Focus on optimizing only critical paths

Avoid side effects in the function being benchmarked.

Run benchmarks multiple times and average the results for accuracy.

Thank you.
Questions?

“Learn from yesterday, live for today, hope for tomorrow. The important thing is not to stop questioning.”

- Albert Einstein